

One instruction per row. Colour tracks where data goes (numbers where the value matters).
Flipping CSR 0x7c0 rotates each spatial op 90° so a pixel is free to move anywhere on the grid.
(vadd / vmv / vmerge are element-wise, so H = V.)

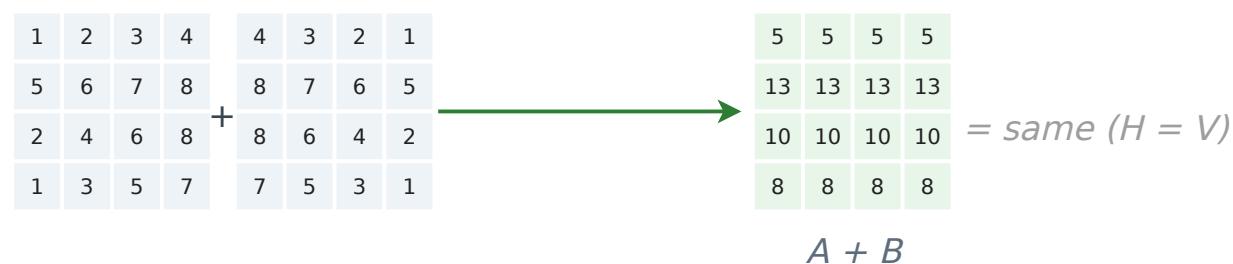
One instruction per row. Colour tracks where data goes (numbers where the value matters).
Flipping CSR 0x7c0 rotates each spatial op 90° so a pixel is free to move anywhere on the grid.
(vadd / vmv / vmerge are element-wise, so H = V.)

vmv.v.x (broadcast)
vmv.v.x v8, a0

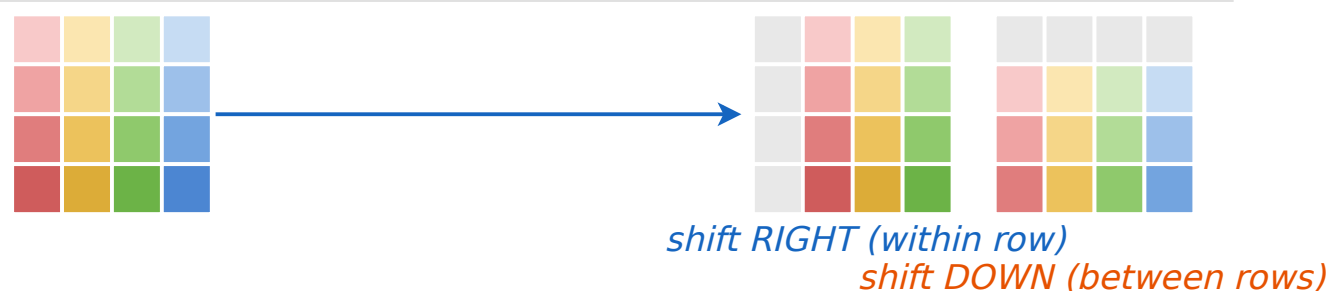
Copy one scalar into every lane.



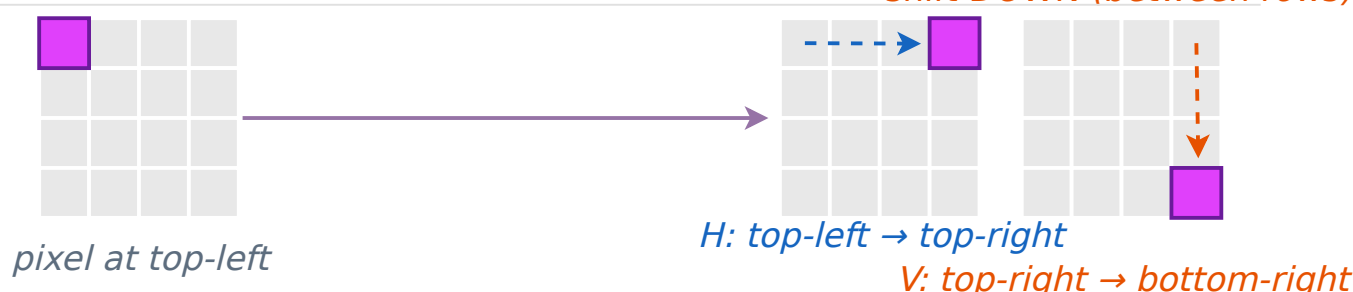
Element-wise add.
Position never
moves.



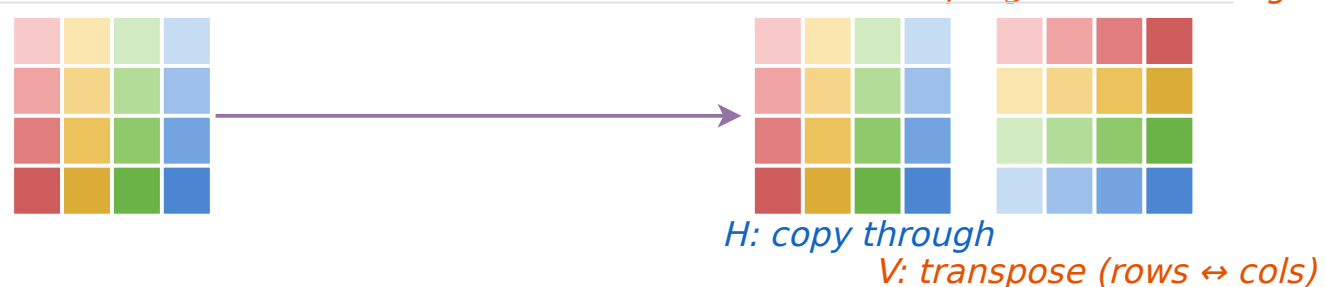
Shift by one. Mode picks the axis.



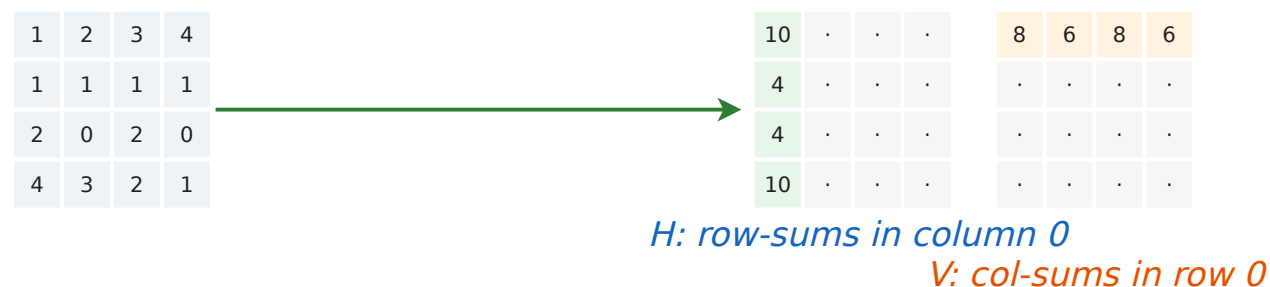
Read any lane $\rightarrow$ any lane. Pixels are not local!



Vertical load/store
transposes the
image.



vredsum.vs
vmv.v.i v0,0
vredsum.vs v8, vs,
x0
Add up. Mode picks
the axis the sum
lands on.



vmerge.vvm + mask v0
vmerge.vvm v8, vB, vA,
v0
 Predicate per lane.
 Pick A where
 mask=1, else B.

